

DOM

INFO

Lesinhoud

DOM (Document Object Model) en DOM-manipulatie.

Doelstellingen

De student(e) kan:

- een webpagina dynamisch maken met behulp van JavaScript (JS) en het Document Object Model (DOM);
- de basisprincipes van JavaScript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM-manipulatie, events en event handling) toepassen binnen een project;
- zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Het DOM \(Document Object Model\)](#)
- [Elementen selecteren uit het DOM](#)
 - [Een element selecteren op basis van id](#)
 - [Elementen selecteren op basis van class](#)
 - [Elementen selecteren op basis van tagnaam](#)
 - [De moderne manier: `querySelector` en `querySelectorAll`](#)
- [Elementen aanmaken](#)
- [Elementen op de pagina plaatsen](#)

- Een element aan het einde toevoegen: `appendChild()`
 - Een element op een specifieke plek invoegen: `insertBefore()`
 - Elementen manipuleren
 - Inhoud van een element aanpassen: `innerText` en `innerHTML`
 - Attributen aanpassen
 - Stijl en classes aanpassen
-

Het DOM (Document Object Model)

Wanneer een browser een HTML-document laadt, zet hij deze om in een boomstructuur van objecten. Deze boomstructuur wordt het Document Object Model, of kortweg DOM, genoemd. Elke tak in de boom is een HTML-element (zoals `body`, `h1`, `p`), en elke tak kan weer eigen takken (andere elementen) bevatten. Deze elementen die in een ander element genest liggen noemen we ook wel child-elements. Het element waar een child-element zich in bevindt noemen we het parent-element.

Je kan het DOM zien als een hiërarchische boomstructuur van je webpagina, waarin elk HTML-element wordt voorgesteld als een object dat JavaScript kan lezen en aanpassen. Dit wijzigen van het DOM met JavaScript noemen we DOM-manipulatie.

Elementen selecteren uit het DOM

Voordat je een element kan aanpassen, moet je het eerst "vinden" of selecteren in het DOM. JavaScript biedt hiervoor verschillende methoden.

Een element selecteren op basis van id

Bekijk de [MDN Web Docs](#).

De meest specifieke manier om één uniek element te vinden is via het `id`-attribuut. Omdat een id uniek moet zijn op een pagina, geeft `getElementById()` altijd maar één element terug (of `null` als het niet gevonden wordt).

```
<h1 id="hoofdtitel">Welkom</h1>
```

html

```
const titel = document.getElementById("hoofdtitel");
titel.innerText = "Welkom op onze website!";
```

js

Welkom op onze website!

Elementen selecteren op basis van class

Bekijk de [MDN Web Docs](#).

Met `getElementsByClassName()` selecteer je alle elementen die een bepaalde `class`-naam hebben. Deze methode geeft een `HTMLCollection` terug, wat lijkt op een array. Je kan er dus overheen itereren met een `for`-loop.

```
<p class="paragraaf">Dit is de eerste paragraaf.</p>
<p class="paragraaf">Dit is de tweede paragraaf.</p>
```

html

```
const paragrafen =
document.getElementsByClassName("paragraaf");

const eersteParagraaf = paragrafen[0]; // Je kan []
gebruiken om een specifiek element uit de HTMLCollection
```

js

te halen.

```
for (let i = 0; i < paragrafen.length; i++) {  
  paragrafen[i].style.color = "blue"; // Maak alle  
  paragrafen blauw.  
}
```

Dit is de eerste paragraaf.

Dit is de tweede paragraaf.

TIP

Merk op hoe bij `getElementById()` het woord `element` in het enkelvoud staat. Hieruit kun je afleiden dat je maar 1 element als resultaat zult krijgen. Kijk nu naar `getElementsByClassName()`. Hier staat `elements` in het meervoud. Hieruit kan je afleiden dat je van deze methode een lijst (`HTMLCollection`) zal terugkrijgen als resultaat. Ook als er maar één element is met de `class` die je als parameter hebt meegegeven. Deze regel geldt ook voor `getElementsByTagName()`.

spreadoperator

Je kan een `HTMLCollection` omzetten naar een echte array met de `spread-operator` (`...`):

```
const paragrafen =  
[...document.getElementsByClassName("paragraaf")];  
  
/**  
 * Het is nu mogelijk om array-methoden zoals .map() te  
 * gebruiken.  
 */  
const gewijzigdeParagrafen = paragrafen.map((p) => {  
  p.style.color = "blue"; // Maak alle paragrafen blauw.
```

js

```
    return p;  
  });
```

Elementen selecteren op basis van tagnaam

Bekijk de [MDN Web Docs](#).

Net als bij `class` -namen, kan je met `getElementsByName()` alle elementen van een bepaald type selecteren (zoals alle `li` - of `p` - elementen). Ook dit geeft een `HTMLCollection` als resultaat.

```
<ul>  
  <li>Item 1</li>  
  <li>Item 2</li>  
</ul>
```

html

```
const listItems = document.getElementsByTagName("li");  
console.log(`Er zijn ${listItems.length} li's op de  
pagina.`); // Geeft "Er zijn 2 li's op de pagina."  
  
const tweedeListItem = listItems[1]; // Je kan []  
gebruiken om een specifiek element uit de HTMLCollection  
te halen.  
  
for (let i = 0; i < listItems.length; i++) {  
  listItems[i].style.textDecoration = "underline"; //  
  Onderlijn alle tekst in de li's in de listItems  
  HTMLCollection.  
}
```

js

- Item 1
- Item 2

De moderne manier: `querySelector` en `querySelectorAll`

De meest krachtige en flexibele manier om elementen te selecteren is met `querySelector()` en `querySelectorAll()`. Deze methoden gebruiken `CSS-selectors` als argument, dezelfde die je gebruikt om je pagina te stijlen.

- `querySelector()`: Geeft het eerste element terug dat voldoet aan de CSS-selector.
- `querySelectorAll()`: Geeft alle elementen terug die voldoen aan de CSS-selector, in de vorm van een `NodeList`.

Een `NodeList` lijkt sterk op een array en je kan er makkelijk overheen itereren met `forEach()` of een gewone `for-loop`.

Bekijk de `querySelector()` [MDN Web Docs](#)

Bekijk de `querySelectorAll()` [MDN Web Docs](#)

```
html
<div id="container">
  <p class="highlight">Eerste paragraaf in de div.</p>
  <p>Tweede paragraaf.</p>
</div>
<p class="highlight">Derde paragraaf buiten de div.</p>
```

```
js
// Selecteer het eerste element met de class 'highlight'.
const eersteHighlight =
document.querySelector(".highlight");
eersteHighlight.style.backgroundColor = "yellow";

// Selecteer alle paragrafen binnen de div met id
'container'.
const paragrafenInContainer =
document.querySelectorAll("#container p");
```

```
// Loop over alle elementen die door de
querySelectorAll()-methode in de pagina gevonden werden.
// In de parameter pElement zit de eerste loop het eerste
element in de paragrafenInContainer NodeList, volgende
loop het tweede, ...
paragrafenInContainer.forEach((pElement) => {
  pElement.style.border = "1px solid black"; // Geef de
  paragraaf die deze loop in pElement zit een zwarte rand.
});
```

Eerste paragraaf in de div.

Tweede paragraaf.

Derde paragraaf buiten de div.

Ook hier kan je de `spread-operator` (`...`) gebruiken om een echte array te maken van de `NodeList`.

```
const paragrafenArray =
[...document.querySelectorAll("p")];
const alleParagrafenInMain = paragrafenArray.filter((p)
=> p.closest("main")); // Filtert alle paragrafen die in
een <main>-element staan.
```

js

Merk op dat bovenstaand filteren als voorbeeld getoond wordt, dit zou ook in één stap kunnen met een CSS-selector:

```
const alleParagrafenInMain =
document.querySelectorAll("main p");
```

js

Elementen aanmaken

Bekijk de [MDN Web Docs](#).

Om via je script een HTML-element te creëren gebruik je de `createElement()`-methode. Deze methode krijgt als parameter de `tag` van het type element dat je wilt aanmaken als string mee. Let wel op, dit element staat na het uitvoeren van de methode nog nergens op de pagina. Waar het element op de pagina moet verschijnen bepaal je later pas.

```
const nieuweTitel = document.createElement("h1");
```

js

Elementen op de pagina plaatsen

Bekijk de `appendChild()` [MDN Web Docs](#) Bekijk de `insertBefore()` [MDN Web Docs](#)

Eens je een nieuw element hebt aangemaakt met `document.createElement()`, is het nog niet zichtbaar op de pagina. Je moet het expliciet toevoegen aan het DOM. Hiervoor zijn `appendChild()` en `insertBefore()` de meestgebruikte methoden.

Een element aan het einde toevoegen: `appendChild()`

De `appendChild()`-methode is de meest eenvoudige manier om een element toe te voegen. Het voegt het nieuwe element toe als het allerlaatste `child-element` van een geselecteerd `parent-element`.

Stappenplan:

- Selecteer/zoek het `parent-element` waar je een element aan wil toevoegen.
- Creëer het nieuwe element.
- Gebruik `parentElement.appendChild(nieuwElement)` om het toe te voegen.

```
<div id="takenlijst">
  <h2>Mijn Taken</h2>
  <ul>
    <li>De les nakijken</li>
  </ul>
</div>
```

html

```
// Selecteer de <ul> als parent-element
const lijst = document.querySelector("#takenlijst ul");

// Creëer een nieuw <li>-element
const nieuweTaak = document.createElement("li");
nieuweTaak.innerText = "De planten water geven"; // Geef
de <li> inhoud

// Voeg het nieuwe element toe aan het einde van de <ul>
lijst.appendChild(nieuweTaak);

/*
Na het uitvoeren van de code ziet de HTML er zo uit:
<ul>
  <li>De les nakijken</li>
  <li>De planten water geven</li>
</ul>
*/
```

js

Mijn Taken

- De les nakijken
- De planten water geven

prepend

Er is ook een modernere methode genaamd `prepend()`, waarmee een element als eerste child van een parent-element kan toevoegd worden.

Een element op een specifieke plek invoegen:

`insertBefore()`

Bekijk de [MDN Web Docs](#).

Soms wil je een element niet aan het einde, maar op een specifieke positie invoegen, bijvoorbeeld helemaal vooraan in een lijst of net vóór een ander element. Hiervoor gebruik je de `insertBefore()`-methode.

Deze methode heeft twee argumenten nodig:

- **Het nieuwe element** dat je wilt invoegen.
- **Het referentie-element**: het child-element waarvóór je het nieuwe element wilt plaatsen.

De methode wordt, net als `appendChild()`, aangeroepen op het parent-element.

```
parentElement.insertBefore(nieuwElement, referentieElement);
```

```
<ol id="top-drie">
  <li id="tweede-plaats">Zilver</li>
```

html

```
<li id="derde-plaats">Brons</li>
</ol>
```

js

```
// Selecteer het parent-element
const topDrieLijst = document.querySelector("#top-drie");

// Creëer het nieuwe element voor de eerste plaats
const eerstePlaats = document.createElement("li");
eerstePlaats.innerText = "Goud";

// Selecteer het referentie-element (het child-element
// waar we het nieuwe element voor willen plaatsen)
const tweedePlaats = document.querySelector("#tweede-
plaats");

// Voeg 'eerstePlaats' in vóór 'tweedePlaats'
topDrieLijst.insertBefore(eerstePlaats, tweedePlaats);

/*
Resultaat:
<ol id="top-drie">
  <li>Goud</li>
  <li id="tweede-plaats">Zilver</li>
  <li id="derde-plaats">Brons</li>
</ol>
*/
```

1. Goud
2. Zilver
3. Brons

Tip

Als je `null` als tweede argument meegeeft aan `insertBefore(nieuwElement, null)`, gedraagt de methode zich exact hetzelfde als `appendChild()` en voegt het element aan het einde toe.

Elementen manipuleren

Als je eenmaal één of meerdere elementen hebt geselecteerd, kan je er van alles mee doen.

Inhoud van een element aanpassen: `innerText` en `innerHTML`

Bekijk de `innerText` [MDN Web Docs](#) Bekijk de `innerHTML` [MDN Web Docs](#)

- `innerText`: Verandert enkel de tekstinhoud van een element. HTML-tags worden als gewone tekst weergegeven. Dit is de veiligste optie.
- `innerHTML`: Kan zowel tekst als HTML-structuur interpreteren en toevoegen. Gebruik dit als je bewust HTML-elementen wilt toevoegen.

`innerHTML` - XSS-risico

`innerHTML` kan een beveiligingsrisico vormen als je input van gebruikers (zoals via een formulier) rechtstreeks op de pagina plaatst. Dit zou kunnen leiden tot Cross-Site Scripting (XSS)-aanvallen.

Om dit te voorkomen, moet je de gebruikersinvoer `sanitize` voordat je deze toevoegt aan de pagina. Indien het niet de bedoeling is dat HTML-tags worden geïnterpreteerd, gebruik dan altijd `innerText` in plaats van `innerHTML`.

html

```
<div id="text-content"></div>
<div id="inner-text"></div>
<div id="inner-html"></div>
```

js

```
const textContentDiv = document.querySelector("#text-content");
const innerTextDiv = document.querySelector("#inner-text");
const innerHtmlDiv = document.querySelector("#inner-html");

textContentDiv.textContent =
  "<h2>Nieuwe titel met een <strong>belangrijk</strong> woord.</h2>";

innerTextDiv.innerText =
  "<h2>Nieuwe titel met een <strong>belangrijk</strong> woord.</h2>";

// Oppassen als de inhoud van een gebruiker komt!
innerHtmlDiv.innerHTML =
  "<h2>Nieuwe titel met een <strong>belangrijk</strong> woord.</h2>";
```

```
<h2>Nieuwe titel met een <strong>belangrijk</strong> woord.</h2>
<h2>Nieuwe titel met een <strong>belangrijk</strong> woord.</h2>
```

Nieuwe titel met een belangrijk woord.

textContent vs innerText

Er is ook nog een derde eigenschap die je kan gebruiken om de tekstinhoud van een element aan te passen, namelijk `textContent`. Het verschil tussen `textContent` en `innerText` is dat `textContent` alle tekst binnen een

element teruggeeft, inclusief verborgen tekst (bijvoorbeeld tekst binnen een element met `display: none`), terwijl `innerText` alleen de zichtbare tekst teruggeeft.

Attributen aanpassen

Je kunt `HTML-attributen` zoals bijvoorbeeld `src` (van een `img` -element), `href` (van een `a` -element) of class direct aanpassen. Begin met een nieuw element te creëren of een bestaand element te zoeken in het DOM. Vervolgens kan je via de `.` het attribuut kiezen die je een nieuwe waarde wilt geven.

```
<a id="mijn-link" href="">
  <img height="100" src="" alt="">
</a>
```

html

```
const link = document.querySelector("#mijn-link");
const afbeelding = link.querySelector("img");

link.href = "https://www.pxl.be";
afbeelding.src = "/assets/img/logo_pxl_digital.png";
afbeelding.alt = "PXL Logo";
```

js



Stijl en classes aanpassen

Bekijk de [MDN Web Docs](#).

Je kunt de stijl van een element direct aanpassen via de `style`-eigenschap. Via `.style` kun je alle CSS-eigenschappen van een element wijzigen. Voor grotere stijlwijzigingen is het echter beter om `CSS-classes` te gebruiken en deze via JavaScript toe te voegen, te verwijderen of te wisselen. Dit kan heel wat regels code uitsparen en houdt logica en opmaak beter gescheiden, wat op zijn beurt de code makkelijker begrijpbaar en onderhoudbaar maakt.

```
const melding = document.createElement("p");

// Inline-stijl aanpassen (niet de beste aanpak voor
// grote wijzigingen)
melding.style.padding = "20px";
melding.style.backgroundColor = "lightblue";

// Classes beheren met classList (betere aanpak voor
// grote wijzigingen)
melding.classList.add("succes"); // Voegt de class
'succes' toe
melding.classList.remove("verborgen"); // Verwijdert de
class 'verborgen'
melding.classList.toggle("actief"); // Voegt 'actief' toe
als die er niet is, of verwijdert hem als die er wel is.
```

js

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)

